

PRÁTICA 04 – CONSULTAS NO MONGODB COM STREAMLIT

Implementação e Gerenciamento de Bancos de Dados NoSQL

Código no GitHub: github.com/joseniltoncirne/Implementacao-e-Geren-de-banco-de-dados/tree/main/pratica04-streamlit

O que é

O OpenF1 Data Explorer é uma página que abre no navegador e mostra os dados de Fórmula 1 que o Exercício 01 guardou no MongoDB (banco openf1_data). Em vez de escrever consultas, a pessoa só escolhe nos menus e a página busca os dados no banco e mostra tudo em números, gráfico e tabela. O trabalho é dividido em 4 arquivos:

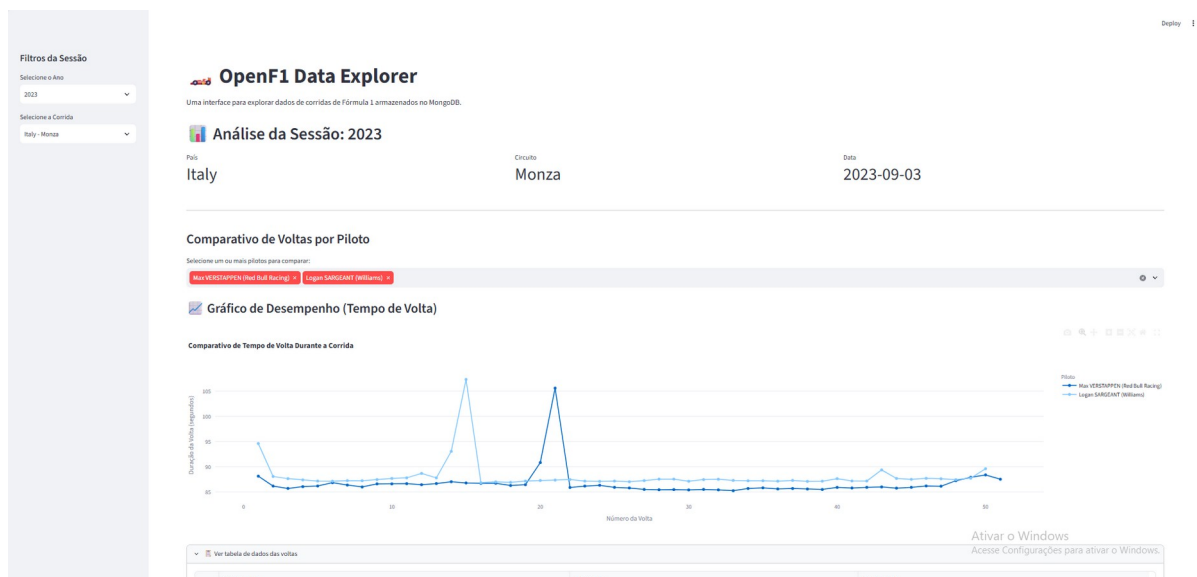
- **streamlit_app.py**: a tela, com os menus, o gráfico e a tabela.
- **db_utils.py**: a parte que conversa com o MongoDB e faz as consultas.
- **requirements.txt**: a lista do que precisa instalar.
- **.env**: o endereço do MongoDB.

Usei o código do professor e só fiz três ajustes pequenos: a lista de anos agora mostra só os anos que existem no banco (antes a página abria em 2025 com dados de exemplo inventados), o menu mostra só as corridas (sem os treinos) e o nome da corrida aparece como "Italy - Monza" em vez de "Corrida Desconhecida".

1. Instalando e abrindo a página

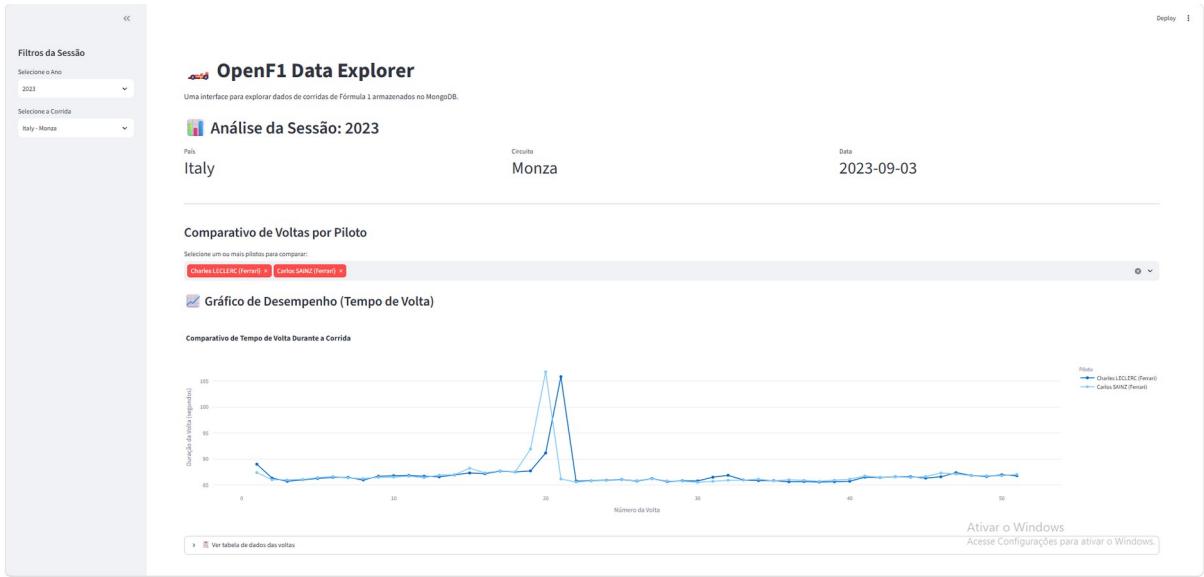
```
python -m pip install -r requirements.txt
python -m streamlit run streamlit_app.py
```

A página abre no navegador já no GP da Itália de 2023, mostrando o país (Italy), o circuito (Monza) e a data da corrida (03/09/2023). O gráfico mostra o tempo de cada uma das 51 voltas dos pilotos escolhidos.



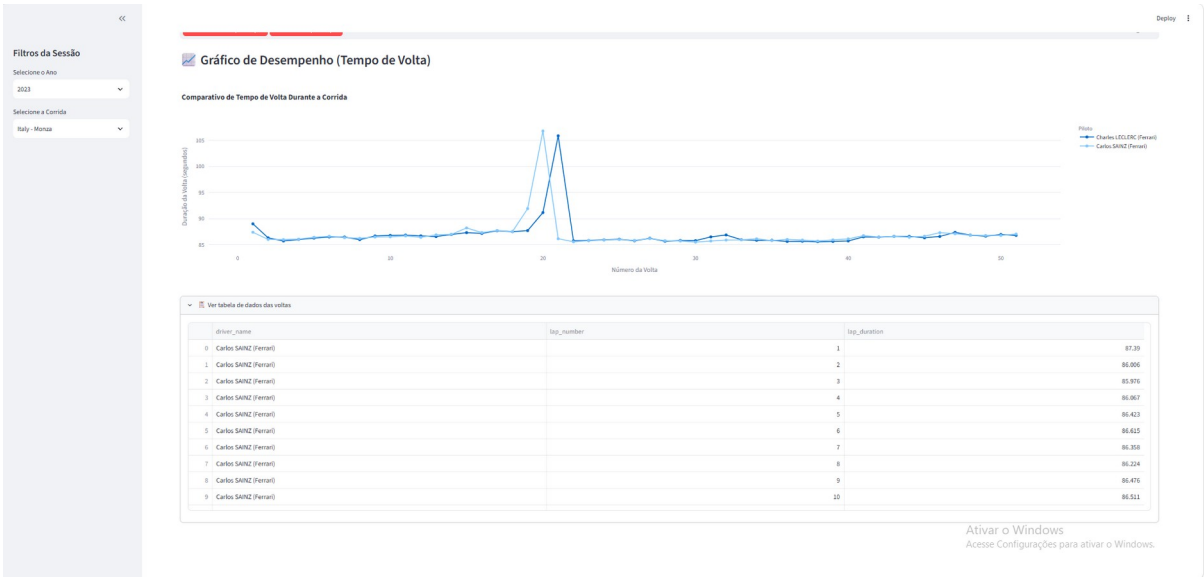
2. Comparando os pilotos da Ferrari

Seguindo o estudo de caso do enunciado, escolhi os dois pilotos da Ferrari, Charles Leclerc e Carlos Sainz. Dá para ver que os dois andaram quase juntos a corrida inteira, com voltas entre 85 e 88 segundos. Os picos perto da volta 20 são as paradas nos boxes: o Sainz parou uma volta antes do Leclerc.



3. Conferindo os números na tabela

Abrindo "Ver tabela de dados das voltas" aparecem os tempos exatos de cada volta, os mesmos que estão guardados no MongoDB.



Código

streamlit_app.py

```
import streamlit as st
import pandas as pd
import plotly.express as px
from db_utils import (
    get_mongo_db,
    get_available_years,
    get_race_sessions,
    get_session_details,
    get_drivers_from_session,
    get_laps_for_drivers
)

st.set_page_config(
    page_title="OpenF1 Data Explorer",
    page_icon="🏎️",
    layout="wide"
)

st.title("🏎️ OpenF1 Data Explorer")
st.markdown("Uma interface para explorar dados de corridas de Fórmula 1 armazenados no MongoDB.")

db = get_mongo_db()

if db is None:
    st.error("❌ Não foi possível conectar ao MongoDB. Verifique a string de conexão e as configurações de rede.")
    st.stop()

with st.sidebar:
    st.header("Filtros da Sessão")

    available_years = get_available_years(db)
    selected_year = st.selectbox("Selecione o Ano", options=available_years)

    race_sessions = get_race_sessions(db, selected_year)
    if not race_sessions:
        st.warning(f"⚠️ Nenhuma sessão de corrida encontrada para {selected_year}.")
        st.stop()

    session_options = {
        f"{s.get('race_name') or s.get('country_name', 'Corrida Desconhecida')} - {s.get('circuit_short_name') or s.get('session_name', 'Sessão')}"
        : s['session_key']
        for s in race_sessions
    }

    selected_race_name = st.selectbox(
        "Selecione a Corrida",
        options=list(session_options.keys())
    )
    selected_session_key = session_options[selected_race_name]

if selected_session_key:
    session_details = get_session_details(db, selected_session_key)
    if session_details:
        st.header(f"📊 Análise da Sessão: {session_details.get('year', 'N/A')}")
        col1, col2, col3 = st.columns(3)
        col1.metric("País", session_details.get('country_name', 'N/A'))
        col2.metric("Circuito", session_details.get('circuit_short_name', 'N/A'))
        col3.metric("Data", session_details.get('date_start', 'N/A').split('T')[0])

        st.markdown("---")

        st.subheader("Comparativo de Voltas por Piloto")
        drivers = get_drivers_from_session(db, selected_session_key)

        if not drivers:
            st.warning("⚠️ Nenhum piloto encontrado para esta sessão.")
            st.stop()
```

```

        driver_map = {
            f"{d.get('full_name', 'Desconhecido')} ({d.get('team_name', 'Equipe N/A')})":
d['driver_number']
            for d in drivers
        }

        selected_drivers_names = st.multiselect(
            "Selecione um ou mais pilotos para comparar:",
            options=list(driver_map.keys()),
            default=list(driver_map.keys())[:2]
        )

        if not selected_drivers_names:
            st.info("❗ Por favor, selecione pelo menos um piloto para visualizar os dados.")
            st.stop()

        selected_driver_numbers = [driver_map[name] for name in selected_drivers_names]

        laps_df = get_laps_for_drivers(db, selected_session_key, selected_driver_numbers)

        if laps_df.empty:
            st.warning("⚠ Não foram encontrados dados de voltas para os pilotos selecionados nesta sessão.")
        else:
            driver_number_to_name = {v: k for k, v in driver_map.items()}
            laps_df['driver_name'] = laps_df['driver_number'].map(driver_number_to_name)

            laps_df = laps_df.dropna(subset=['lap_duration'])
            laps_df = laps_df[laps_df['lap_duration'] > 0]

            st.subheader("📊 Gráfico de Desempenho (Tempo de Volta)")
            fig = px.line(
                laps_df,
                x="lap_number",
                y="lap_duration",
                color="driver_name",
                title="Comparativo de Tempo de Volta Durante a Corrida",
                labels={
                    "lap_number": "Número da Volta",
                    "lap_duration": "Duração da Volta (segundos)",
                    "driver_name": "Piloto"
                },
                markers=True
            )
            st.plotly_chart(fig, width="stretch")

            with st.expander("📄 Ver tabela de dados das voltas"):
                st.dataframe(
                    laps_df[['driver_name', 'lap_number', 'lap_duration']]
                    .sort_values(by=['driver_name', 'lap_number'])
                    .reset_index(drop=True)
                )

```

db_utils.py

```

import os
import random
import pandas as pd
from datetime import datetime
from pymongo import MongoClient
from pymongo.errors import ConnectionFailure
from dotenv import load_dotenv

load_dotenv()

def get_mongo_db():
    mongo_uri = os.getenv("MONGO_URI", "mongodb://localhost:27017")
    db_name = os.getenv("MONGO_DB", "openfl_data")

    try:
        client = MongoClient(mongo_uri, serverSelectionTimeoutMS=2000)
        client.admin.command("ping")
        return client[db_name]
    except ConnectionFailure:

```

```

        print("⚠ Não foi possível conectar ao MongoDB. Usando dados mockados.")
        return None

def get_available_years(db):
    anos = sorted(db.sessions.distinct("year"), reverse=True) if db is not None else []
    return anos or [2025, 2024, 2023]

def get_race_sessions(db, year):
    if db is None:
        return _mock_get_race_sessions(year)

    sessions = list(db.sessions.find({"year": int(year), "session_name": "Race"}, {"_id": 0}))
    if not sessions:
        return _mock_get_race_sessions(year)
    return sessions

def get_session_details(db, session_key):
    if db is None:
        return _mock_get_session_details(session_key)

    session = db.sessions.find_one({"session_key": session_key}, {"_id": 0})
    if not session:
        return _mock_get_session_details(session_key)
    return session

def get_drivers_from_session(db, session_key):
    if db is None:
        return _mock_get_drivers_from_session(session_key)

    drivers = list(db.drivers.find({"session_key": session_key}, {"_id": 0}))
    if not drivers:
        return _mock_get_drivers_from_session(session_key)
    return drivers

def get_laps_for_drivers(db, session_key, driver_numbers):
    if db is None:
        return _mock_get_laps_for_drivers(session_key, driver_numbers)

    laps = list(
        db.laps.find(
            {"session_key": session_key, "driver_number": {"$in": driver_numbers}},
            {"_id": 0}
        )
    )
    if not laps:
        return _mock_get_laps_for_drivers(session_key, driver_numbers)
    return pd.DataFrame(laps)

def _mock_get_race_sessions(year):
    return [
        {"session_key": f"{year}_AUS_GP", "race_name": "GP da Austrália", "session_name": "Corrida"},
        {"session_key": f"{year}_BRA_GP", "race_name": "GP do Brasil", "session_name": "Corrida"},
        {"session_key": f"{year}_MON_GP", "race_name": "GP de Mônaco", "session_name": "Corrida"},
    ]

def _mock_get_session_details(session_key):
    return {
        "year": session_key.split("_")[0],
        "country_name": "Brasil" if "BRA" in session_key else "Austrália" if "AUS" in session_key else "Mônaco",
        "circuit_short_name": "Interlagos" if "BRA" in session_key else "Albert Park" if "AUS" in session_key else "Monte Carlo",
        "date_start": datetime.now().strftime("%Y-%m-%dT%H:%M:%S"),
    }

```

```
def _mock_get_drivers_from_session(session_key):
    return [
        {"driver_number": 1, "full_name": "Max Verstappen", "team_name": "Red Bull"},
        {"driver_number": 16, "full_name": "Charles Leclerc", "team_name": "Ferrari"},
        {"driver_number": 44, "full_name": "Lewis Hamilton", "team_name": "Mercedes"},
        {"driver_number": 63, "full_name": "George Russell", "team_name": "Mercedes"},
    ]

def _mock_get_laps_for_drivers(session_key, driver_numbers):
    laps_data = []
    for driver in driver_numbers:
        base_time = random.uniform(85, 95)
        for lap in range(1, 31):
            lap_duration = base_time + random.uniform(-2, 2)
            laps_data.append({
                "driver_number": driver,
                "lap_number": lap,
                "lap_duration": lap_duration
            })
    return pd.DataFrame(laps_data)
```

requirements.txt

```
streamlit
pymongo
python-dotenv
pandas
plotly
```

.env

```
MONGO_URI="mongodb://localhost:27017"
```